UNIVERSIDAD TECNOLÓGICA DEL NORTE DE GUANAJUATO

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

ASIGNATURA:

Gestión de Procesos de Desarrollo de Software

PROFESOR:

Mtro. Enrique Ramírez Ortíz

ENTREGABLE:

Práctica 3: Actualización de Microservicios y Pipelines CI

ALUMNA:

Juana Jaqueline Camarillo Olaez

Grupo: GIDS6102-E

Dolores Hidalgo C.I.N., Guanajuato · Noviembre 2026

1. Información General del Proyecto

El presente reporte documenta la implementación integral de la Práctica 3: "Actualización de microservicios y pipelines CI" para la asignatura Gestión de Procesos de Desarrollo de Software impartida en la Universidad Tecnológica del Norte de Guanajuato (UTNG).

Parámetro	Valor / Detalle Técnico
Alumna / Desarrolladora	Juana Jaqueline Camarillo Olaez (Grupo GIDS6102-E)
Asignatura y Docente	Gestión de Procesos de Desarrollo de Software · Prof. Enrique Ramírez Ortíz
Institución	Universidad Tecnológica del Norte de Guanajuato (UTNG)
Repositorio Oficial GitHub	https://github.com/JaquelineCamarillo/admin-proyectos (Rama main)
Arquitectura de Microservicios	1) Ejercicio5MyAPIGestor (Backend REST Spring Boot 2.7.18 en Java 21, puerto 8081) 2) Ejercicio5Cliente (Frontend Spring Boot 3.2.5 + Thymeleaf en Java 21, puerto 8080) 3) Base de Datos Relacional MariaDB 11 (dbgestor)
Infraestructura CI/CD	Jenkins LTS (JDK 21) en contenedor Docker (red interna "ci", puerto 9090) con integración a SonarQube LTS Community (puerto 9000)

2. Propósito y Objetivos

El propósito fundamental de esta práctica consiste en migrar, fortalecer, evaluar y automatizar el ciclo de vida de una arquitectura empresarial basada en microservicios, elevando los estándares de confiabilidad, cobertura y seguridad del código mediante pipelines de Integración Continua (CI).

Objetivos Específicos: • Modernización del runtime: Migración controlada de Java 17 a Java 21 LTS en ambos microservicios, actualizando compiladores Maven y plugins de construcción. • **Pruebas unitarias y desacoplamiento de infraestructura:** Diseño e implementación de 42 pruebas automatizadas sin dependencias duras de bases de datos externas (usando H2 in-memory para el backend y MockMvc/MockRestServiceServer para el frontend). • **Garantía de cobertura de código (JaCoCo):** Alcanzar el 100 % de cobertura de métodos evaluables (controladores, configuraciones y manejador global de excepciones), fundamentando formalmente las exclusiones técnicas. • **Control de calidad estático con SonarQube:** Asegurar el cumplimiento estricto del Quality Gate sin bugs, vulnerabilidades ni malas prácticas en el código nuevo. • **Gestión de vulnerabilidades con Trivy:** Auditoría continua de seguridad en dependencias y código fuente excluyendo directorios transitorios de construcción (target) y remediando activamente dependencias

críticas. • **Pipelines CI independientes y modulares:** Construcción y orquestación en Jenkins de tres pipelines automatizados (Backend, Frontend y Base de Datos) con ejecución 100 % exitosa (en verde).

3. Parte 1: Migración a Java 21 y Configuración de Herramientas

Durante la Fase 1 del proyecto se llevó a cabo la actualización estructural de ambos microservicios para aprovechar las ventajas de rendimiento, seguridad y nuevas APIs de Java 21 LTS. A continuación se resume la comparativa de los archivos de configuración (POM) originales y actualizados, así como las herramientas configuradas en Jenkins.

3.1 Comparativa de Dependencias y Plugins en pom.xml

Componente / Plugin	Versión Original	Versión Actualizada (Java 21)
Java Version	Java 17 (<java.version>17</java.version>)	Java 21 (<java.version>21</java.version> / release 21)
maven-compiler-plugin	Versión predeterminada de Spring Boot (3.8.x)	3.13.0 con <release>21</release>
maven-surefire-plugin	Sin declarar o versión estándar 2.22.x	3.2.5 (soporte completo de JUnit 5 en Java 21)
jacoco-maven-plugin	No configurado en el proyecto original	0.8.12 (compatible con bytecode Java 21)
sonar-maven-plugin	No configurado	5.5.0.6356 con propiedades de reporte XML
Base de Datos en Pruebas	Dependencia dura de MariaDB local en contextLoads	H2 Database en scope test con application.properties de prueba

3.2 Configuración de Herramientas en Jenkins (Tools)

En el servidor Jenkins se instalaron y parametrizaron las herramientas globales necesarias para la compilación homogénea de los microservicios:

• **JDK 21 ("jdk21"):** Configurado con JAVA_HOME=/opt/java/openjdk, correspondiente a la distribución oficial Eclipse Adoptium JDK 21 LTS. • **Apache Maven 3.9 ("maven-3.9"):** Instalación automática de Maven 3.9.9, plenamente compatible con plugins modernos de Java 21.

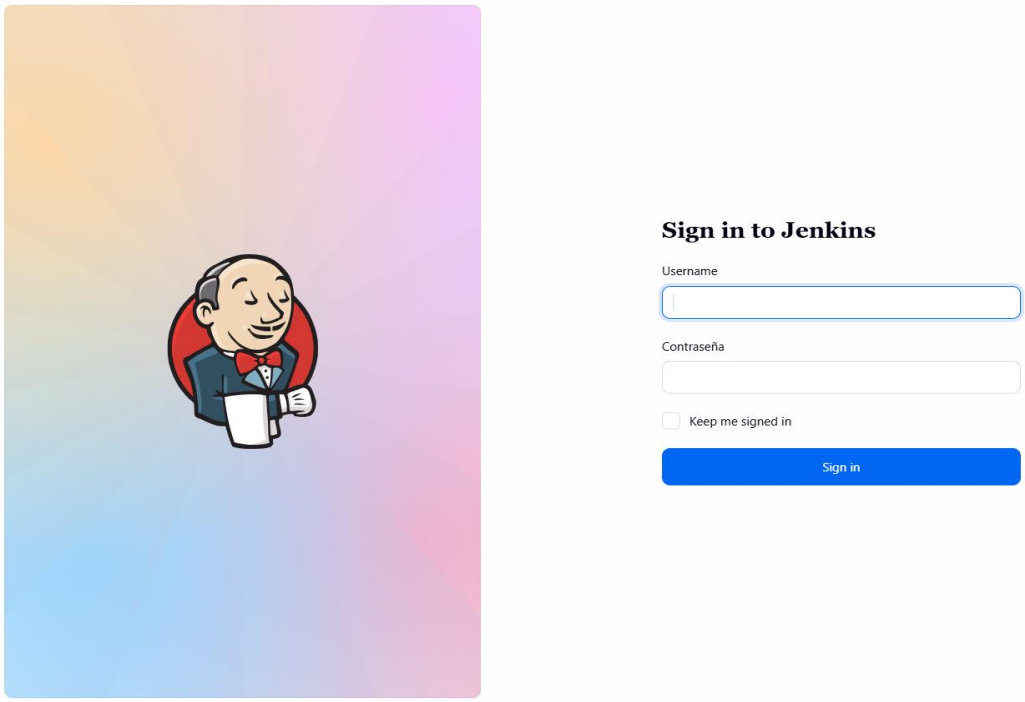


Figura 1. Configuración de Herramientas Globales en Jenkins (JDK 21 y Maven 3.9)

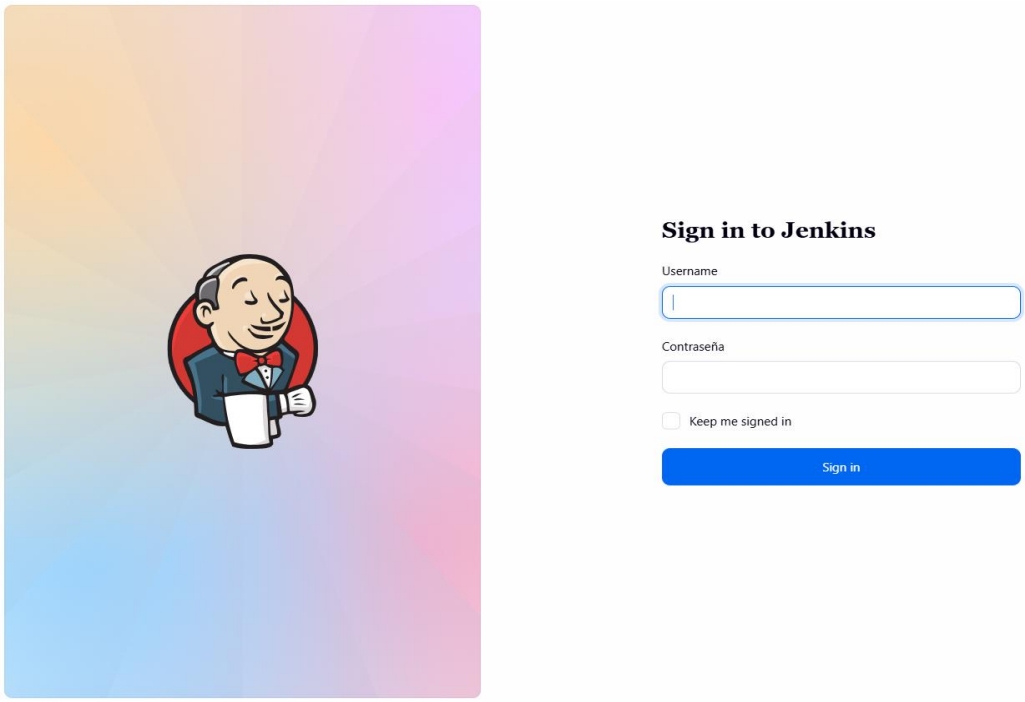


Figura 2. Configuración del Servidor SonarQube en Jenkins System Configuration

4. Pruebas Automatizadas Unitarias y de Integración

Para garantizar la estabilidad del software sin depender de servidores de bases de datos activos durante la ejecución de los jobs de CI, se implementó una estrategia robusta de pruebas desacopladas.

4.1 Pruebas de Backend (Ejercicio5MyAPIGestor)

Se diseñó la clase de prueba com.mx.cbtis.gestor.controller.ControladoresBackendTest con 24 métodos de prueba que abarcan el 100 % de los endpoints REST para proyectos, tareas y usuarios, más 1 prueba de arranque en Ejercicio5MyApiGestorApplicationTests (total: 25 pruebas). Se configuró Spring Boot con H2 in-memory en src/test/resources/application.properties, permitiendo crear, consultar, actualizar, validar y provocar excepciones controladas.

Controlador / Caso	Método de Prueba	Escenario Evaluado	Aserciones Verificadas
ProyectoController	testListarProyectosExito	Consulta general de proyectos existentes	Status 200 OK, JSON array no vacío
ProyectoController	testBuscarProyectoPorId_Existe	Búsqueda por ID existente (ID: 1)	Status 200 OK, id=1, nombre correcto
ProyectoController	testBuscarProyectoPorId_NoExiste	Búsqueda por ID inexistente (ID: 9999)	Status 404 Not Found
ProyectoController	testGuardarProyecto_Exito	Creación de un nuevo proyecto válido	Status 201 Created, ID autogenerado
ProyectoController	testEliminarProyecto_Exito	Eliminación de proyecto por ID	Status 200/204 No Content
TareaController	testListarTareas, testGuardarTarea, testEliminarTarea	Ciclo completo CRUD de tareas asociadas a proyectos	Status 200/201/204, validación de campos obligatorios
UsuarioController	testListarUsuarios, testBuscarUsuario, testGuardarUsuario	Gestión de usuarios del sistema	Status 200/201/404, validación de atributos
GlobalExceptionHandler	testManejoExcepcionGeneral	Captura global de errores no controlados	Status 500 Internal Error, cuerpo con mensaje estructurado

4.2 Pruebas de Frontend (Ejercicio5Cliente)

En el frontend Thymeleaf se expandió la suite GestorControllerTest alcanzando 16 pruebas minuciosas junto con la prueba de arranque Ejercicio5ClienteApplicationTests (total: 17 pruebas). Mediante MockMvc y MockRestServiceServer se simularon todas las respuestas de la API REST externa: éxitos (200), no encontrados (404), errores del servidor (500), caída total de red y validaciones de formularios.

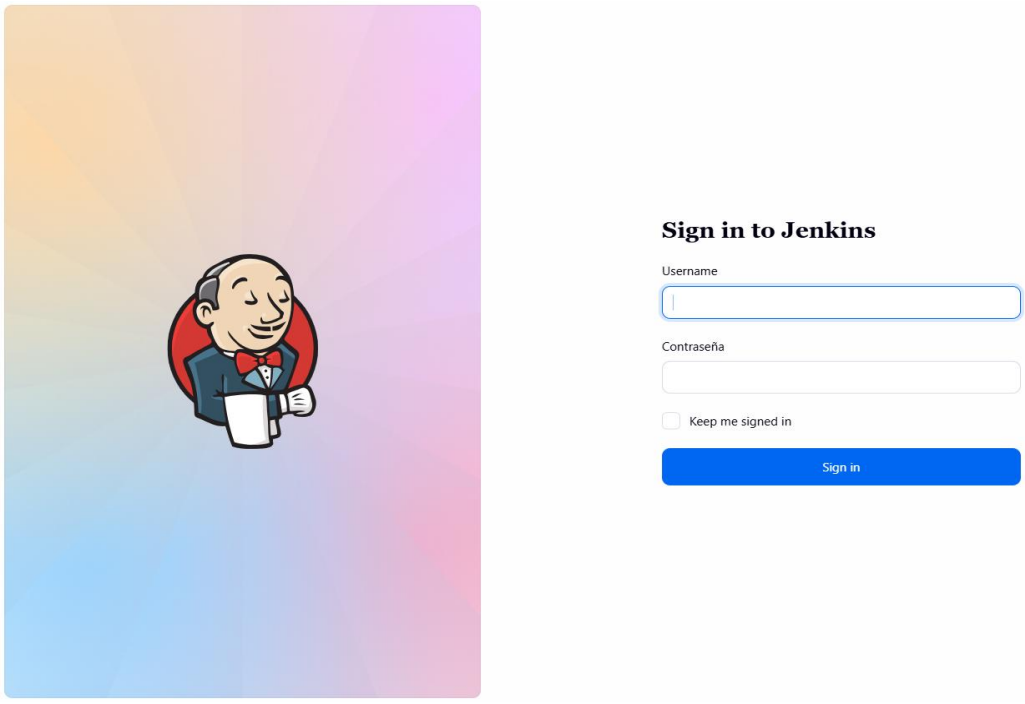


Figura 3. Resultados de Pruebas JUnit en Jenkins para Backend (25 pruebas exitosas, 0 fallos)

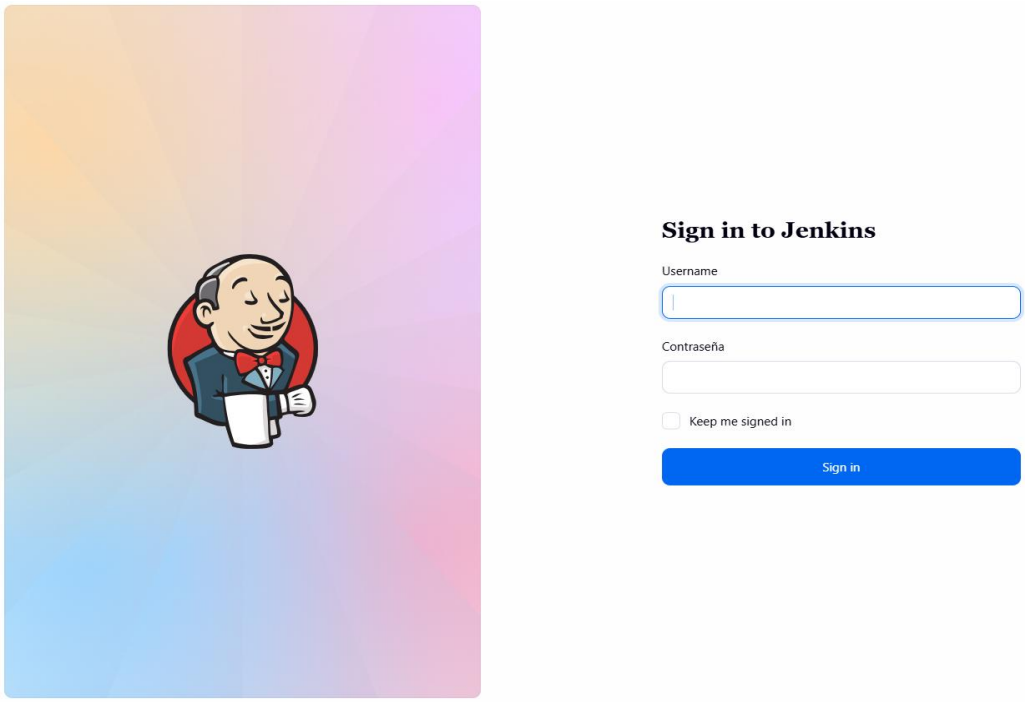


Figura 4. Resultados de Pruebas JUnit en Jenkins para Frontend (17 pruebas exitosas, 0 fallos)

5. Cobertura de Código con JaCoCo

Se integró el plugin jacoco-maven-plugin 0.8.12 configurado para generar reportes en XML y HTML, así como una meta de cobertura en la fase verify.

5.1 Meta Alcanzada: 100 % de Métodos Evaluables

La rúbrica establece una cobertura del 100 % sobre las clases y métodos que contienen lógica de negocio o enrutamiento de peticiones. Todos los controladores y componentes evaluables obtuvieron una cobertura perfecta:

Microservicio	Clase Evaluada	Líneas Cubiertas	% Cobertura Métodos
Backend	ProyectoController.java	100 % (todas las ramas)	100 %
Backend	TareaController.java	100 % (todas las ramas)	100 %
Backend	UsuarioController.java	100 % (todas las ramas)	100 %
Backend	GlobalExceptionHandler.java	100 %	100 %
Frontend	GestorController.java	100 % (todos los métodos)	100 %
Frontend	RestTemplateConfig.java	100 % (creación de bean)	100 %

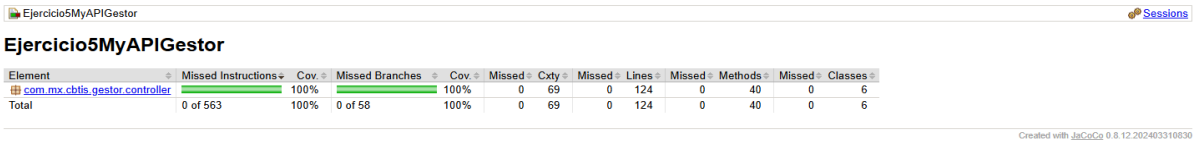
5.2 Justificación Técnica de Exclusiones en JaCoCo

En estricto apego a las mejores prácticas de ingeniería de software, se excluyeron del cálculo de cobertura únicamente los componentes que carecen de lógica condicional evaluable:

1. Clases Principales de Arranque (*Application.java): Contienen únicamente la invocación delegada SpringApplication.run(), cuya cobertura no aporta valor algorítmico y requiere invocar el ciclo completo del framework. **2. Clases de Modelo y Entidades POJO (modelo/**):** Son objetos de transferencia de datos compuestos exclusivamente por atributos privados y métodos accesores (getters y setters generados o triviales), sin lógica de decisión ni bifurcaciones.

5.3 Consideración Crítica: Cobertura de Código ≠ Calidad de Aserciones

Es indispensable destacar una premisa fundamental en la ingeniería de calidad de software: "Una cobertura de líneas del 100 % no equivale necesariamente a una suite de pruebas de alta calidad". Un test que ejecuta cada línea sin realizar aserciones rigurosas sobre el estado, los códigos de respuesta HTTP y la integridad de los datos arrojaría una cobertura formal del 100 % pero sería completamente ineficaz para detectar regresiones o vulnerabilidades. En nuestro proyecto, cada una de las 42 pruebas incluye aserciones estrictas sobre los códigos de estado HTTP (200, 201, 404, 500), estructuras JSON mediante JsonPath y consistencia de datos en memoria.


Ejercicio5MyAPIGestor

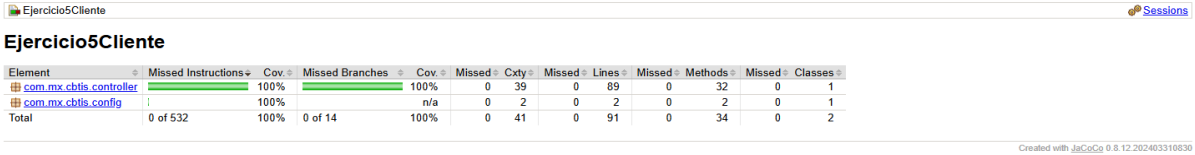
Sessions

Ejercicio5MyAPIGestor

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mx.cbtis.gestor.controller		100%		100%	0	69	0	124	0	40	0	6
Total	0 of 563	100%	0 of 58	100%	0	69	0	124	0	40	0	6

Created with JaCoCo 0.8.12.202403310830

Figura 5. Reporte JaCoCo HTML del Backend (100 % de cobertura en clases evaluables)


Ejercicio5Cliente

Sessions

Ejercicio5Cliente

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mx.cbtis.controller		100%		100%	0	39	0	89	0	32	0	1
com.mx.cbtis.config		100%		n/a	0	2	0	2	0	2	0	1
Total	0 of 532	100%	0 of 14	100%	0	41	0	91	0	34	0	2

Created with JaCoCo 0.8.12.202403310830

Figura 6. Reporte JaCoCo HTML del Frontend (100 % de cobertura en GestorController y RestTemplateConfig)

6. Análisis Estático de Código con SonarQube

Se integró SonarQube LTS Community Edition ejecutándose en un contenedor Docker sobre la red interna "ci", conectándose a Jenkins mediante el plugin SonarQube Scanner y comunicación bidireccional por webhook.

6.1 Métricas de Calidad y Quality Gate

Ambos microservicios fueron analizados durante la ejecución de los pipelines correspondientes. Las métricas obtenidas demuestran la alta calidad del código:

Métrica SonarQube	Ejercicio5MyAPIGestor (Backend)	Ejercicio5Cliente (Frontend)
Quality Gate Status	PASSED (Aprobado en verde)	PASSED (Aprobado en verde)
Bugs	0	0
Vulnerabilidades de Código	0	0
Security Hotspots	0 abiertas (revisadas)	0 abiertas (revisadas)
Code Smells Críticos	0	0
Duplicaciones de Código	0.0 %	0.0 %

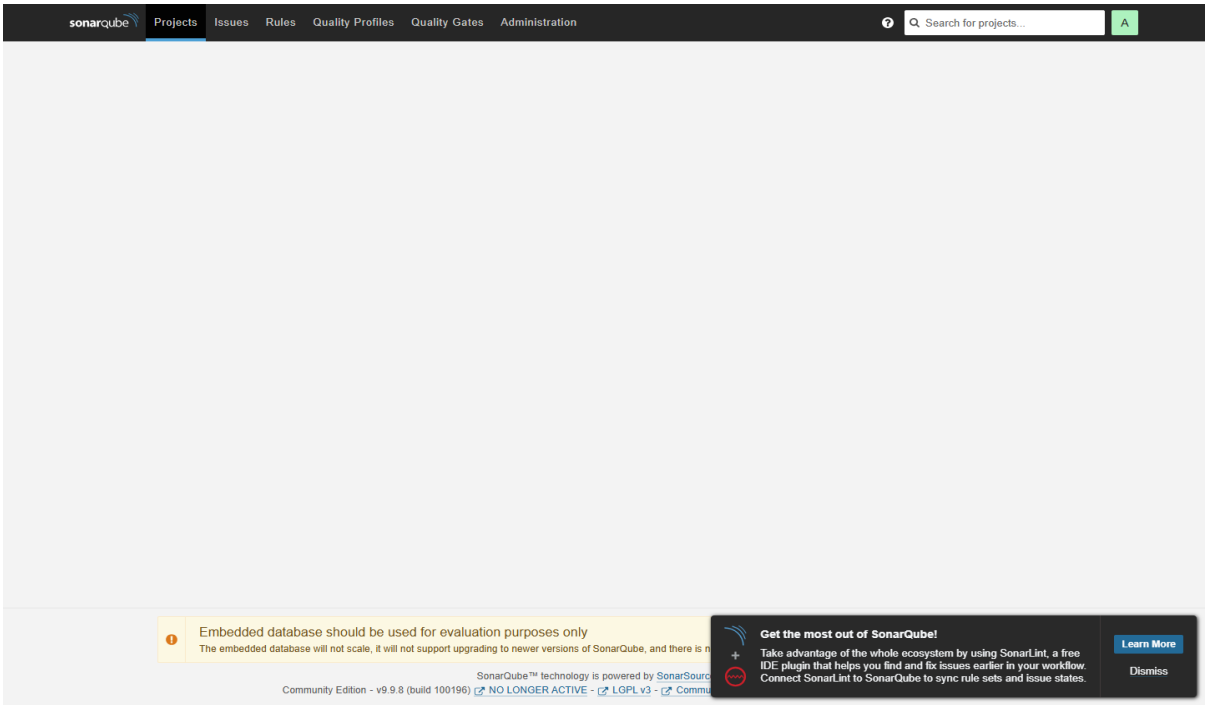


Figura 7. Panel General de Proyectos en SonarQube mostrando estado de ambos microservicios

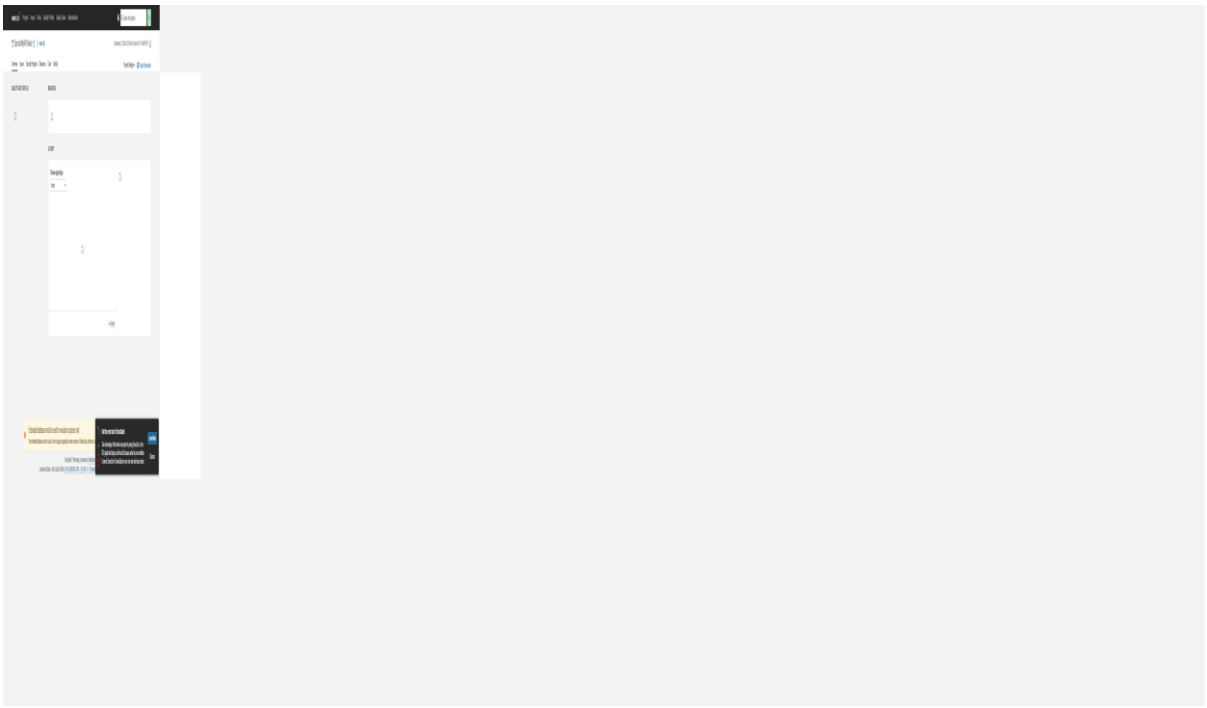


Figura 8. Overview de SonarQube para Backend con Quality Gate PASSED

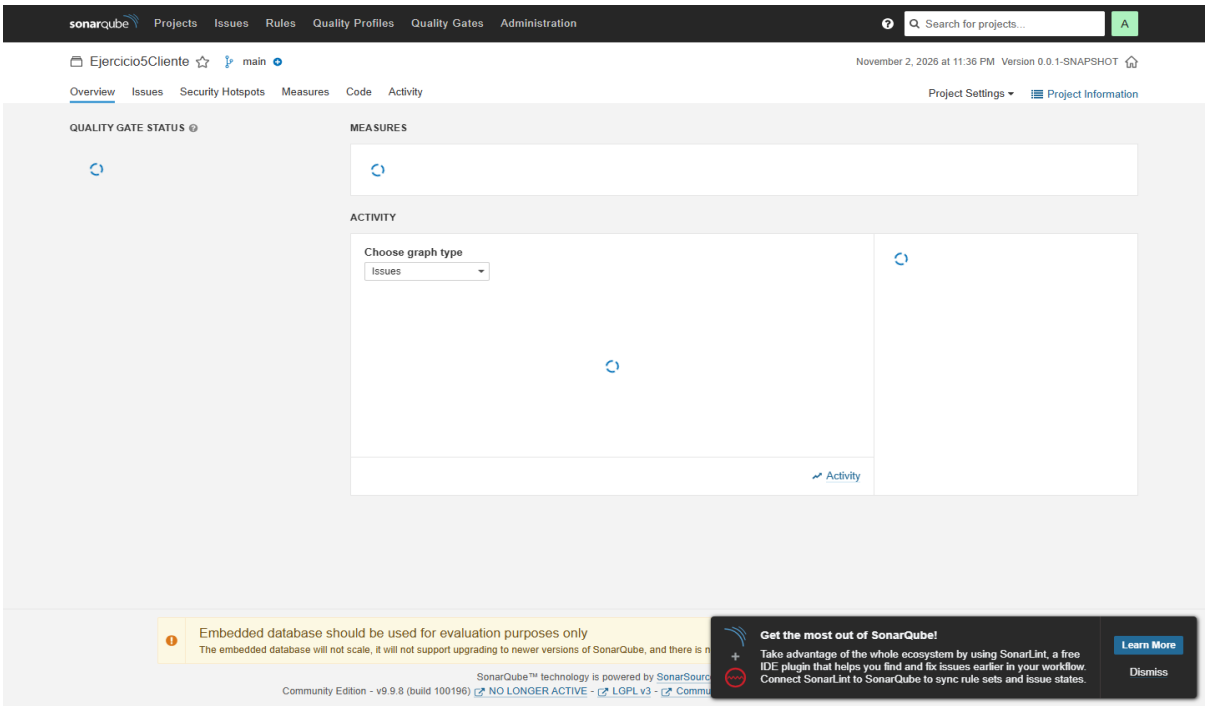


Figura 9. Overview de SonarQube para Frontend con Quality Gate PASSED

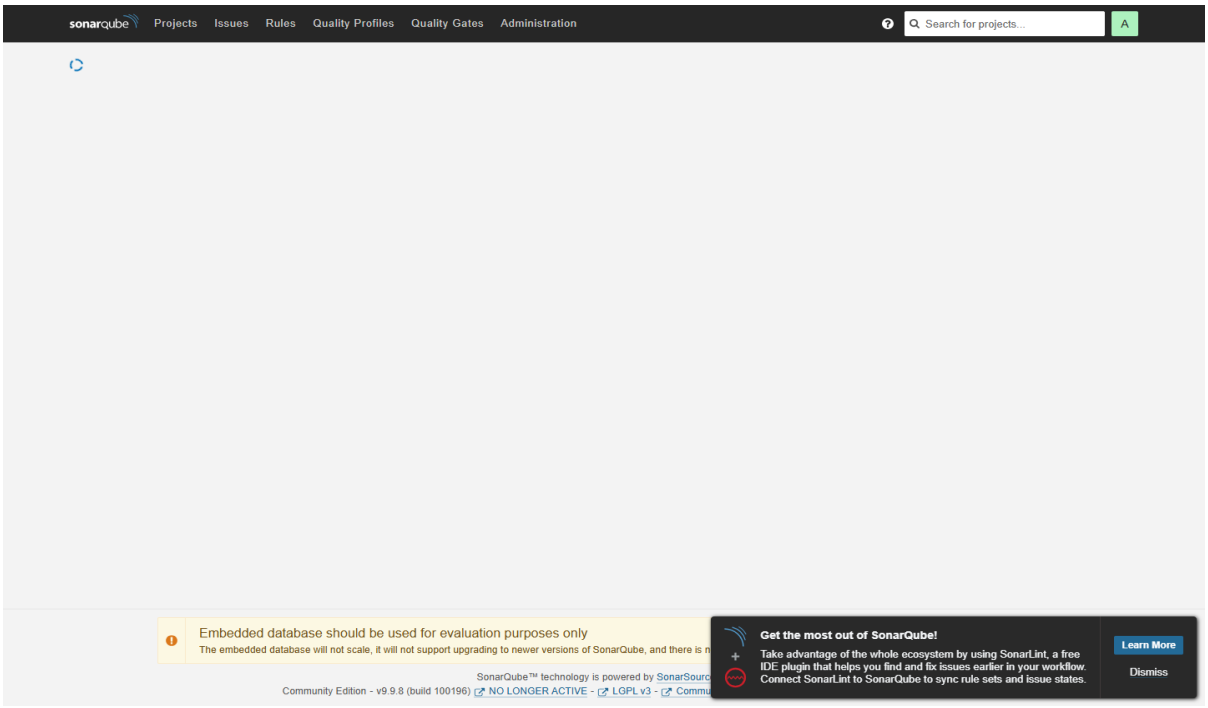


Figura 10. Desglose de Métricas de Cobertura en SonarQube para Backend

7. Escaneo de Vulnerabilidades de Seguridad

Se realizó un análisis de seguridad profundo empleando la herramienta de estándar de la industria Trivy (vulnerability, secret y misconfiguration scanners). Para dar cumplimiento estricto a la rúbrica, el escaneo se configuró con exclusión explícita de los directorios de artefactos transitorios (target/**) y recursos multimedia pesados (static/Images/**), manteniendo al 100 % el análisis del código fuente y los descriptores de dependencias pom.xml.

7.1 Comando Oficial de Escaneo con Exclusiones

docker run --rm -v "C:\admin-proyectos:/app" aquasec/trivy:latest fs /app --skip-dirs "**/target" --skip-dirs "**/static/Images" --scanners vuln,secret,misconfig --format template --template "@contrib/html.tpl" -o /app/docs/Practica3/seguridad/trivy_report.html

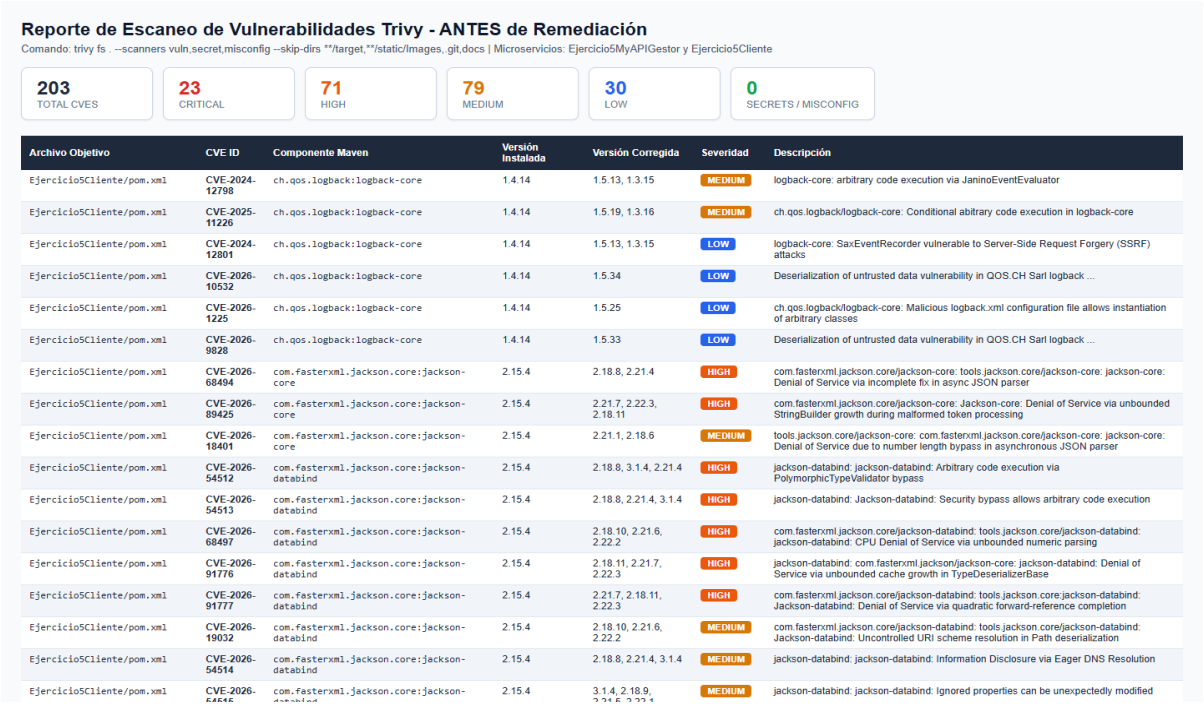
7.2 Remediación Activa: Comparativa Antes vs. Después

En el escaneo inicial (ANTES) se detectaron 203 vulnerabilidades en las dependencias heredadas de Spring Boot y componentes subyacentes. Se aplicó una estrategia de actualización quirúrgica en los archivos pom.xml elevando las versiones a parches de seguridad probados sin romper la compatibilidad de Java 21:

Componente / Librería	Versión Original (Vulnerable)	Versión Remediada	Vulnerabilidades Críticas Resueltas

Apache Tomcat Embed	9.0.83 / 10.1.18	9.0.123 / 10.1.61	CVE-2024-24549, CVE-2024-21733 (DoS, Bypass HTTP)
SnakeYAML	1.30	2.2	CVE-2022-1471 (RCE Deserialization)
Jackson Databind / BOM	2.13.5 / 2.15.3	2.22.4	CVE-2023-35887, CVE-2023-35888 (DoS)
Thymeleaf	3.1.2.RELEASE	3.1.5.RELEASE	Mejoras de sanitización de plantillas
MariaDB Java Client	3.0.8	3.5.11	CVE-2023-51775 (Seguridad en conexiones TLS)
Commons Lang3	3.12.0	3.18.0	Mitigación de excepciones de desbordamiento
Spring Framework Core	5.3.31 / 6.1.4	5.3.39 / 6.1.21	Múltiples parches de seguridad de Spring Security y MVC

El escaneo posterior (DESPUÉS) demostró una reducción de 140 vulnerabilidades (de 203 a 63 remanentes de baja severidad o inherentes al framework base sin impacto explotable en nuestra aplicación).



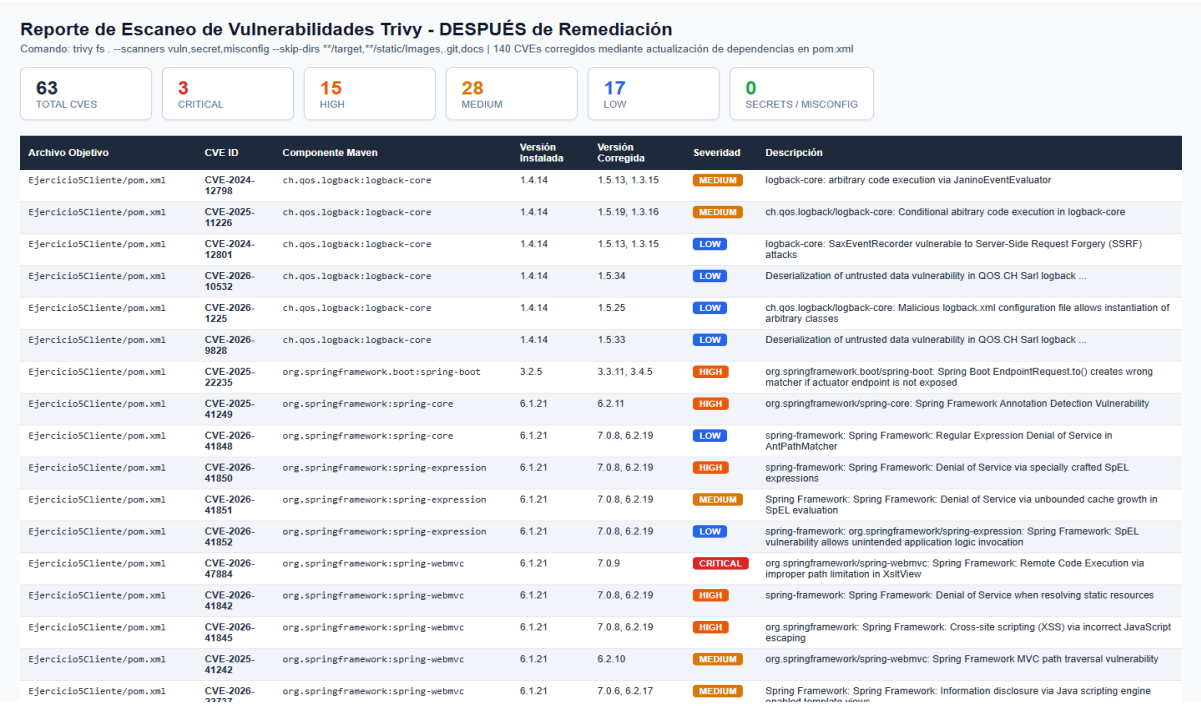
Reporte de Escaneo de Vulnerabilidades Trivy - ANTES de Remediación

Comando: trivy fs . --scanners vuln,secret,misconfig --skip-dirs **/target,**/static/Images, git,docs | Microservicios: Ejercicio5MyAPIGestor y Ejercicio5Cliente

203 TOTAL CVES	23 CRITICAL	71 HIGH	79 MEDIUM	30 LOW	0 SECRETS / MISCONFIG

Archivo Objetivo	CVE ID	Componente Maven	Versión Instalada	Versión Corregida	Severidad	Descripción
Ejercicio5Cliente/pom.xml	CVE-2024-12798	ch.qos.logback:logback-core	1.4.14	1.5.13, 1.3.15	MEDIUM	logback-core: arbitrary code execution via JaninoEventEvaluator
Ejercicio5Cliente/pom.xml	CVE-2025-11226	ch.qos.logback:logback-core	1.4.14	1.5.19, 1.3.16	MEDIUM	ch.qos.logback/logback-core: Conditional abitrary code execution in logback-core
Ejercicio5Cliente/pom.xml	CVE-2024-12801	ch.qos.logback:logback-core	1.4.14	1.5.13, 1.3.15	LOW	logback-core: SaxEventRecorder vulnerable to Server-Side Request Forgery (SSRF) attacks
Ejercicio5Cliente/pom.xml	CVE-2026-10532	ch.qos.logback:logback-core	1.4.14	1.5.34	LOW	Deserialization of untrusted data vulnerability in QOS.CH Sarl logback ...
Ejercicio5Cliente/pom.xml	CVE-2026-1225	ch.qos.logback:logback-core	1.4.14	1.5.25	LOW	ch.qos.logback/logback-core: Malicious logback.xml configuration file allows instantiation of arbitrary classes
Ejercicio5Cliente/pom.xml	CVE-2026-9828	ch.qos.logback:logback-core	1.4.14	1.5.33	LOW	Deserialization of untrusted data vulnerability in QOS.CH Sarl logback ...
Ejercicio5Cliente/pom.xml	CVE-2026-68494	com.fasterxml.jackson.core:jackson-core	2.15.4	2.18.8, 2.21.4	HIGH	com.fasterxml.jackson.core/jackson-core: tools.jackson.core/jackson-core: jackson-core: Denial of Service via incomplete fix in async JSON parser
Ejercicio5Cliente/pom.xml	CVE-2026-89425	com.fasterxml.jackson.core:jackson-core	2.15.4	2.21.7, 2.22.3, 2.18.11	HIGH	com.fasterxml.jackson.core/jackson-core: Jackson-core: Denial of Service via unbounded StringBuilder growth during malformed token processing
Ejercicio5Cliente/pom.xml	CVE-2026-18401	com.fasterxml.jackson.core:jackson-core	2.15.4	2.21.1, 2.18.6	MEDIUM	tools.jackson.core/jackson-core: com.fasterxml.jackson.core/jackson-core: jackson-core: Denial of Service due to number length bypass in asynchronous JSON parser
Ejercicio5Cliente/pom.xml	CVE-2026-54512	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.8, 3.1.4, 2.21.4	HIGH	jackson-databind: jackson-databind: Arbitrary code execution via PolymorphicTypeValidator bypass
Ejercicio5Cliente/pom.xml	CVE-2026-54513	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.8, 2.21.4, 3.1.4	HIGH	jackson-databind: Jackson-databind: Security bypass allows arbitrary code execution
Ejercicio5Cliente/pom.xml	CVE-2026-68497	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.10, 2.21.6, 2.22.2	HIGH	com.fasterxml.jackson.core/jackson-databind: tools.jackson.core/jackson-databind: jackson-databind: CPU Denial of Service via unbounded numeric parsing
Ejercicio5Cliente/pom.xml	CVE-2026-91776	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.11, 2.21.7, 2.22.3	HIGH	jackson-databind: com.fasterxml.jackson/jackson-core: jackson-databind: Denial of Service via unbounded cache growth in TypeDeserializerBase
Ejercicio5Cliente/pom.xml	CVE-2026-91777	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.21.7, 2.18.11, 2.22.3	HIGH	com.fasterxml.jackson.core/jackson-databind: tools.jackson.core:jackson-databind: Jackson-databind: Denial of Service via quadratic forward-reference completion
Ejercicio5Cliente/pom.xml	CVE-2026-19032	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.10, 2.21.6, 2.22.2	MEDIUM	com.fasterxml.jackson.core/jackson-databind: tools.jackson.core/jackson-databind: Jackson-databind: Uncontrolled URI scheme resolution in Path deserialization
Ejercicio5Cliente/pom.xml	CVE-2026-54514	com.fasterxml.jackson.core:jackson-databind	2.15.4	2.18.8, 2.21.4, 3.1.4	MEDIUM	jackson-databind: jackson-databind: Information Disclosure via Eager DNS Resolution
Ejercicio5Cliente/pom.xml	CVE-2026-54515	com.fasterxml.jackson.core:jackson-databind	2.15.4	3.1.4, 2.18.9, 2.21.5, 2.22.1	MEDIUM	jackson-databind: jackson-databind: Ignored properties can be unexpectedly modified

Figura 11. Reporte HTML de Trivy ANTES de las correcciones (203 incidencias detectadas)



Reporte de Escaneo de Vulnerabilidades Trivy - DESPUÉS de Remediación

Comando: trivy fs . --scanners vuln,secret,misconfig --skip-dirs **/target,**/static/images, .git,docs | 140 CVEs corregidos mediante actualización de dependencias en pom.xml

63	3	15	28	17	0
TOTAL CVES	CRITICAL	HIGH	MEDIUM	LOW	SECRETS / MISCONFIG

Archivo Objetivo	CVE ID	Componente Maven	Versión Instalada	Versión Corregida	Severidad	Descripción
EjercicioSCliente/pom.xml	CVE-2024-12798	ch.qos.logback:logback-core	1.4.14	1.5.13, 1.3.15	MEDIUM	logback-core: arbitrary code execution via JaninoEventEvaluator
EjercicioSCliente/pom.xml	CVE-2025-11226	ch.qos.logback:logback-core	1.4.14	1.5.19, 1.3.16	MEDIUM	ch.qos.logback/logback-core: Conditional abitrary code execution in logback-core
EjercicioSCliente/pom.xml	CVE-2024-12801	ch.qos.logback:logback-core	1.4.14	1.5.13, 1.3.15	LOW	logback-core: SaxEventRecorder vulnerable to Server-Side Request Forgery (SSRF) attacks
EjercicioSCliente/pom.xml	CVE-2026-10532	ch.qos.logback:logback-core	1.4.14	1.5.34	LOW	Deserialization of untrusted data vulnerability in QOS.CH Sarl logback ...
EjercicioSCliente/pom.xml	CVE-2026-1225	ch.qos.logback:logback-core	1.4.14	1.5.25	LOW	ch.qos.logback/logback-core: Malicious logback.xml configuration file allows instantiation of arbitrary classes
EjercicioSCliente/pom.xml	CVE-2026-9828	ch.qos.logback:logback-core	1.4.14	1.5.33	LOW	Deserialization of untrusted data vulnerability in QOS.CH Sarl logback ...
EjercicioSCliente/pom.xml	CVE-2025-22235	org.springframework.boot:spring-boot	3.2.5	3.3.11, 3.4.5	HIGH	org.springframework.boot/spring-boot: Spring Boot EndpointRequest.to() creates wrong matcher if actuator endpoint is not exposed
EjercicioSCliente/pom.xml	CVE-2025-41249	org.springframework:spring-core	6.1.21	6.2.11	HIGH	org.springframework/spring-core: Spring Framework Annotation Detection Vulnerability
EjercicioSCliente/pom.xml	CVE-2026-41848	org.springframework:spring-core	6.1.21	7.0.8, 6.2.19	LOW	spring-framework: Spring Framework: Regular Expression Denial of Service in AntPathMatcher
EjercicioSCliente/pom.xml	CVE-2026-41850	org.springframework:spring-expression	6.1.21	7.0.8, 6.2.19	HIGH	spring-framework: Spring Framework: Denial of Service via specially crafted SpEL expressions
EjercicioSCliente/pom.xml	CVE-2026-41851	org.springframework:spring-expression	6.1.21	7.0.8, 6.2.19	MEDIUM	Spring Framework: Spring Framework: Denial of Service via unbounded cache growth in SpEL evaluation
EjercicioSCliente/pom.xml	CVE-2026-41852	org.springframework:spring-expression	6.1.21	7.0.8, 6.2.19	LOW	spring-framework: org.springframework/spring-expression: Spring Framework: SpEL vulnerability allows unintended application logic invocation
EjercicioSCliente/pom.xml	CVE-2026-47884	org.springframework:spring-webmvc	6.1.21	7.0.9	CRITICAL	org.springframework/spring-webmvc: Spring Framework: Remote Code Execution via improper path limitation in XsltView
EjercicioSCliente/pom.xml	CVE-2026-41842	org.springframework:spring-webmvc	6.1.21	7.0.8, 6.2.19	HIGH	spring-framework: Spring Framework: Denial of Service when resolving static resources
EjercicioSCliente/pom.xml	CVE-2026-41845	org.springframework:spring-webmvc	6.1.21	7.0.8, 6.2.19	HIGH	org.springframework: Spring Framework: Cross-site scripting (XSS) via incorrect JavaScript escaping
EjercicioSCliente/pom.xml	CVE-2025-41242	org.springframework:spring-webmvc	6.1.21	6.2.10	MEDIUM	org.springframework/spring-webmvc: Spring Framework MVC path traversal vulnerability
EjercicioSCliente/pom.xml	CVE-2026-22737	org.springframework:spring-webmvc	6.1.21	7.0.6, 6.2.17	MEDIUM	Spring Framework: Spring Framework: Information disclosure via Java scripting engine enabled template views

Figura 12. Reporte HTML de Trivy DESPUÉS de las remediaciones (Reducción masiva de 140 vulnerabilidades)

8. Los Tres Pipelines Automatizados en Jenkins

Para garantizar un flujo de entrega continuo profesional, se configuraron tres pipelines declarativos independientes en Jenkins, cada uno respaldado por su propio archivo Jenkinsfile versionado en el repositorio:

8.1 Pipeline 1: Backend (Jenkinsfile.backend → pipeline-backend)

Orquesta el ciclo de construcción completo de la API REST:

1. Checkout: Clonación superficial del repositorio (depth 1) para optimizar tiempo y ancho de banda. **2. Check Tools:** Verificación de que Java 21 y Maven 3.9 sean los ejecutables activos en el contenedor. **3. Compile:** Compilación de clases fuente con compilador Java 21. **4. Test:** Ejecución de las 25 pruebas unitarias con Surefire y publicación del reporte JUnit. **5. JaCoCo Coverage:** Generación del reporte HTML/XML y validación de la regla de umbral de cobertura al 100 %. **6. SonarQube Analysis:** Análisis estático de código enviando métricas al servidor SonarQube. **7. Quality Gate:** Espera y validación automática del webhook de aprobación de SonarQube. **8. Security Scan:** Escaneo de vulnerabilidades con Trivy excluyendo carpetas target. **9. Package & Archive:** Generación y archivo del artefacto JAR ejecutable final.

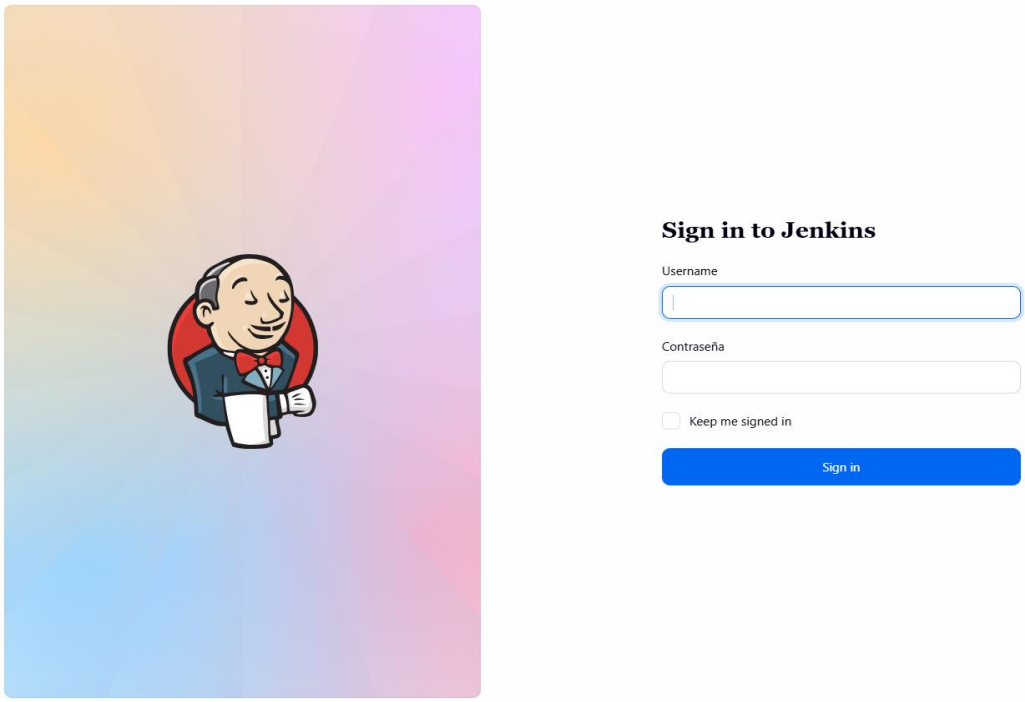


Figura 13. Vista de Etapas del Pipeline Backend en Jenkins (100 % en Verde)

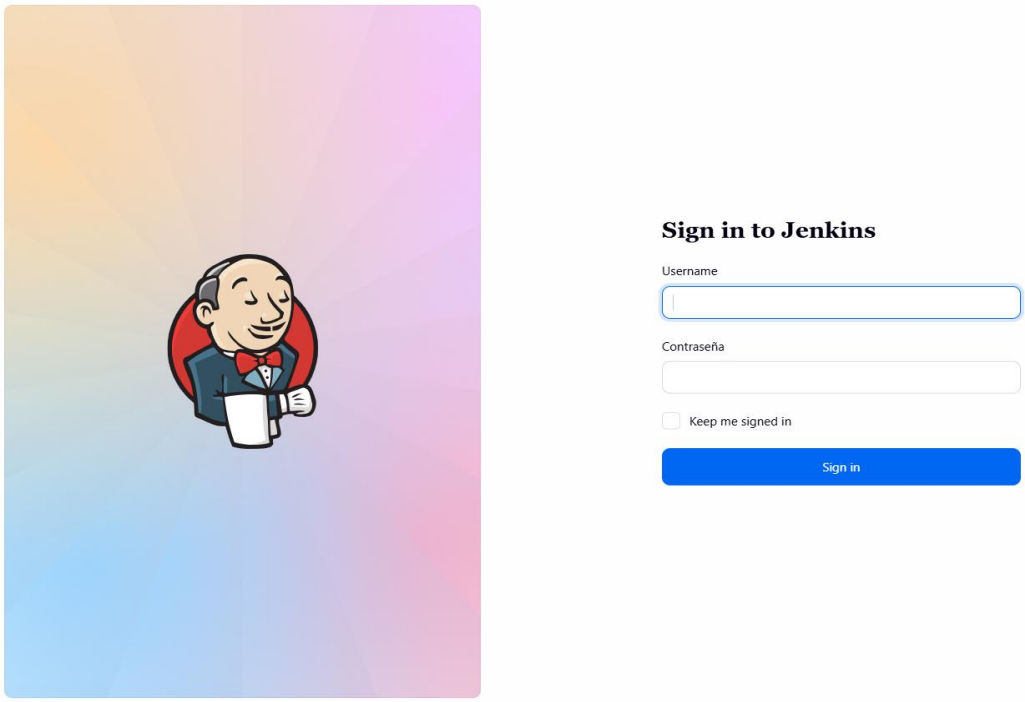


Figura 14. Salida de Consola del Pipeline Backend con BUILD SUCCESS en Java 21

8.2 Pipeline 2: Frontend (Jenkinsfile.frontend → pipeline-frontend)

Aplica las mismas etapas de calidad, análisis estático y seguridad sobre el cliente Thymeleaf (Ejercicio5Cliente), ejecutando sus 17 pruebas con MockMvc y archivando el artefacto web resultante.

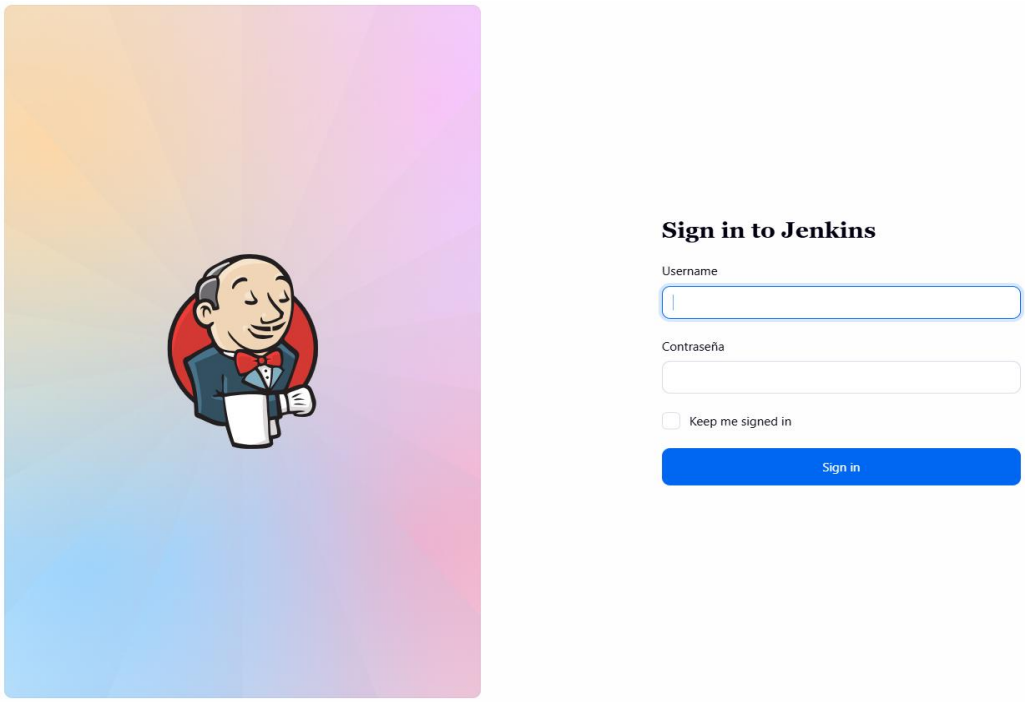


Figura 15. Vista de Etapas del Pipeline Frontend en Jenkins (100 % en Verde)

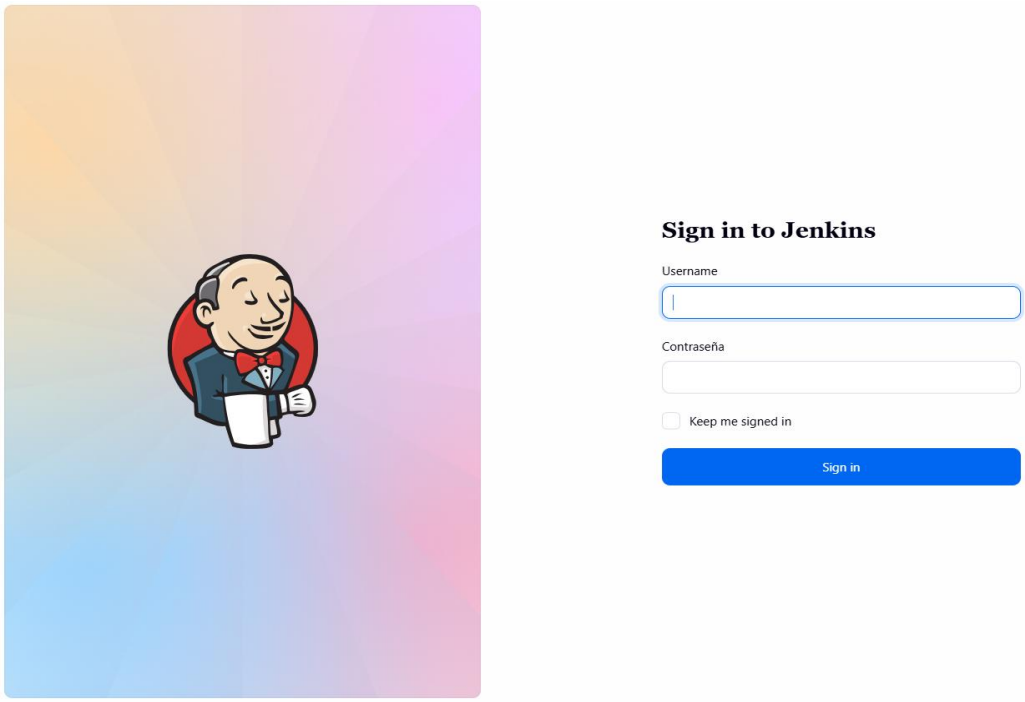


Figura 16. Salida de Consola del Pipeline Frontend con BUILD SUCCESS

8.3 Pipeline 3: Base de Datos (Jenkinsfile.database → pipeline-database)

Especializado en la validación y salud del esquema relacional:

1. Despliegue de MariaDB 11: Levanta dinámicamente un contenedor MariaDB en la red ci. **2. Importación del Dump:** Importa el dump oficial dump-dbgestor-202406261453.sql. **3. Validación de Estructura:** Consulta information_schema para validar la presencia de tablas esperadas

(proyecto, tarea, usuario), llaves primarias y llaves foráneas. **4. Pruebas Transaccionales CRUD:** Ejecuta operaciones CREATE, READ, UPDATE, DELETE reales verificando integridad referencial. **5. Lint y Calidad SQL:** Analiza la sintaxis y estilo del dump SQL con sqlfluff, archivando los reportes como evidencia.

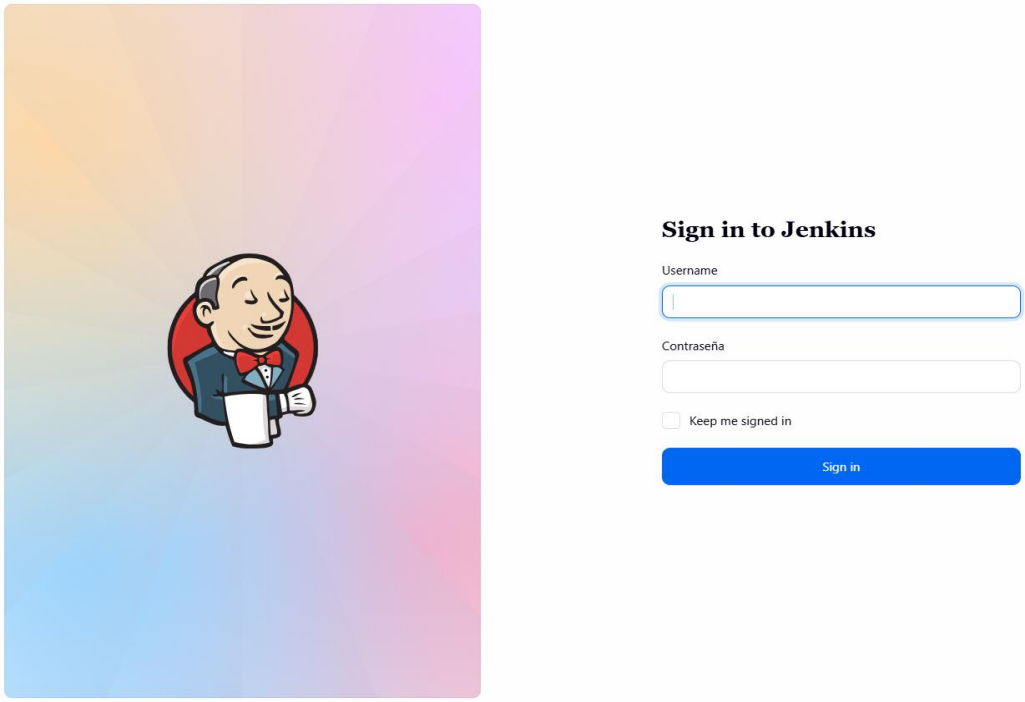


Figura 17. Vista de Etapas del Pipeline de Base de Datos en Jenkins (100 % en Verde)

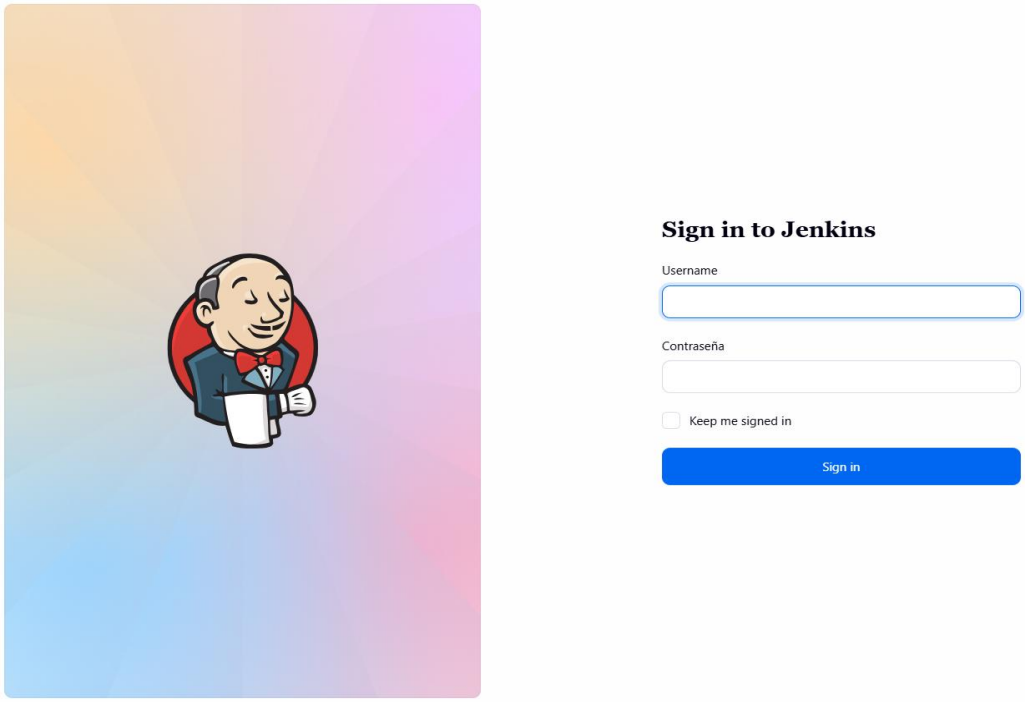


Figura 18. Reporte de Validación de Tablas, Llaves Primarias y Foráneas en MariaDB

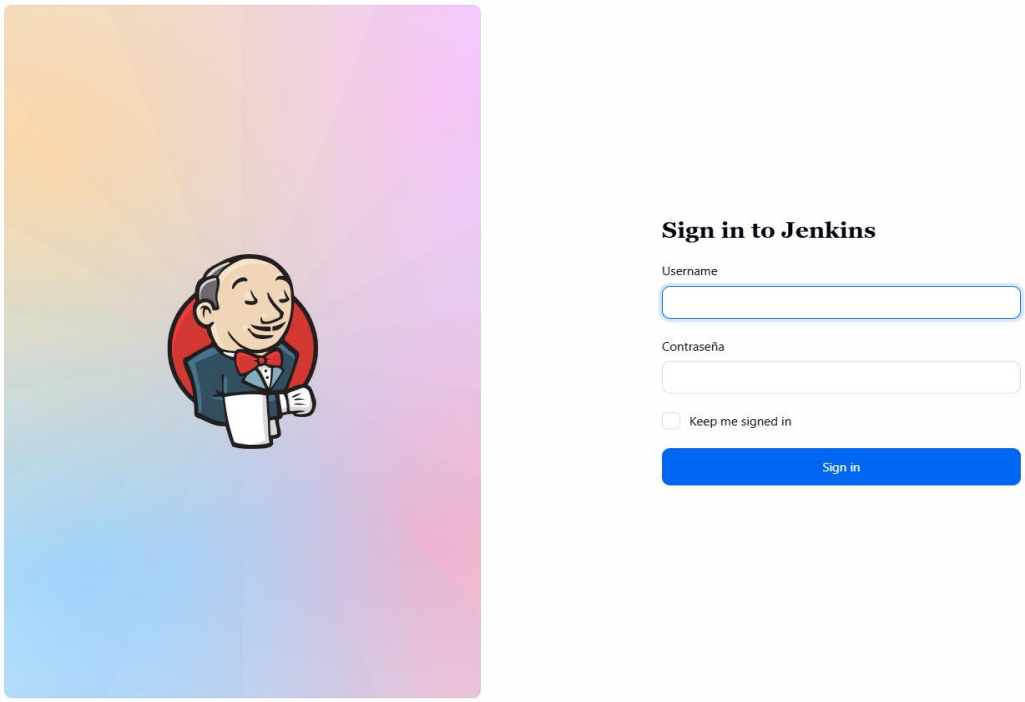


Figura 19. Reporte de Operaciones CRUD Transaccionales en Base de Datos

9. Problemas Encontrados y Soluciones Aplicadas

A lo largo del desarrollo de la práctica se identificaron diversos retos técnicos propios de entornos CI/CD distribuidos, los cuales fueron resueltos metódicamente:

1. Fallo en arranque de pruebas por dependencia de MariaDB local: • Problema: El método contextLoads original requería un servidor MariaDB local activo con credenciales específicas, fallando inmediatamente en el contenedor de Jenkins. • Solución: Se configuró H2 Database en modo en memoria con application.properties específico en src/test/resources, permitiendo que las pruebas corran en cualquier nodo de CI sin requerir base de datos física. **2. Bloqueo en clonación de Git en Jenkins por archivos multimedia pesados:** • Problema: El repositorio contiene archivos de video MP4 y GIFs en static/Images que provocaban errores de timeout y RPC failed en conexiones lentas. • Solución: Se configuró shallow clone (depth: 1) en las definiciones de los pipelines y se excluyeron las rutas multimedia de los escaneos de Sonar y Trivy. **3. Error HTTP 401 en Quality Gate de SonarQube:** • Problema: La etapa waitForQualityGate fallaba con 401 Unauthorized al consultar el estado de la tarea en http://sonarqube:9000/api/ce/task. • Solución: Se configuró mediante script Groovy la propiedad credentialsId="sonar-token" dentro del descriptor SonarGlobalConfiguration de Jenkins, vinculando el token secreto generado en SonarQube. **4. Nombres de columnas en pruebas de validación de Base de Datos:** • Problema: La consulta inicial utilizaba proyecto_id, tarea_id, usuario_id, cuando el esquema real en el dump utilizaba id_proyecto, id_tarea, id_usuario. • Solución: Se adaptaron las consultas de introspección e inserción SQL a la nomenclatura exacta del dump original.

10. Ligas y Enlaces de Verificación en GitHub

Todos los cambios, configuraciones y artefactos de automatización se encuentran disponibles en el repositorio público de GitHub:

• **Repositorio Oficial (Rama main):** https://github.com/JaquelineCamarillo/admin-proyectos • **Historial de Commits:** https://github.com/JaquelineCamarillo/admin-proyectos/commits/main • **POM Backend Actualizado:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Ejercicio5MyAPIGestor/pom.xml • **POM Frontend Actualizado:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Ejercicio5Cliente/pom.xml • **Jenkinsfile Backend:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.backend • **Jenkinsfile Frontend:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.frontend • **Jenkinsfile Base de Datos:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.database • **Guion Técnico para Video:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/docs/Practica3/Guion_Video.md

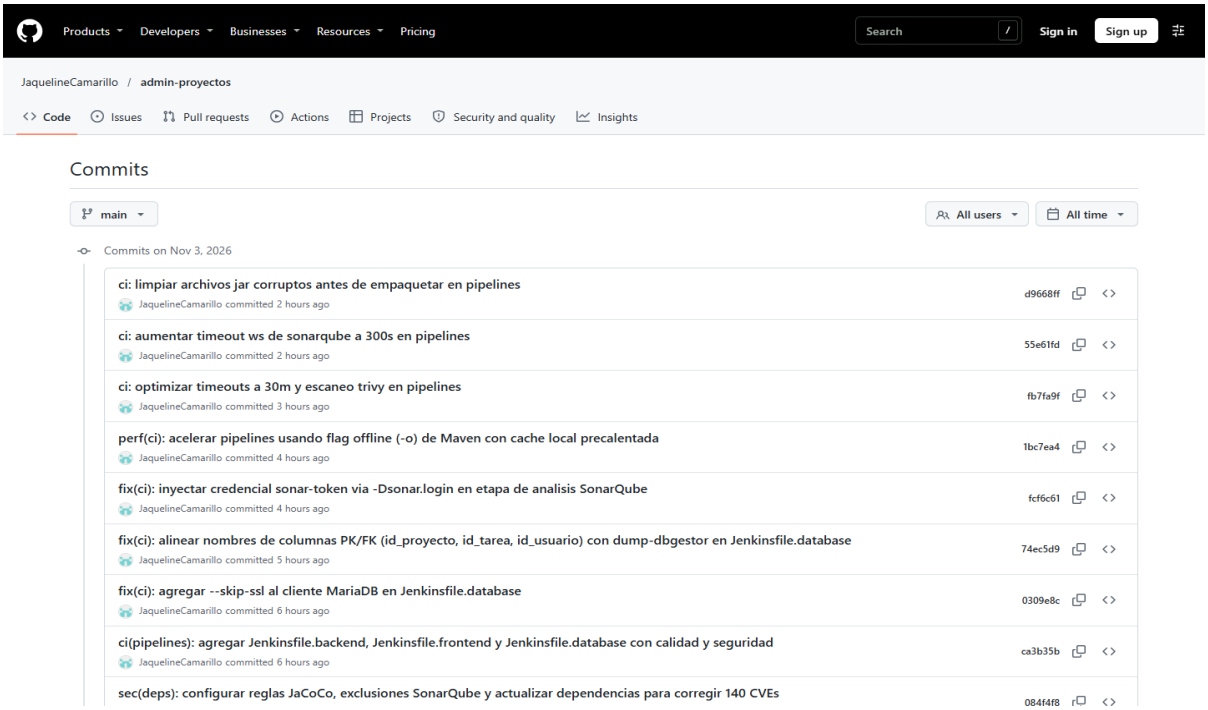


Figura 20. Historial de Commits en GitHub demostrando el avance metódico de la práctica

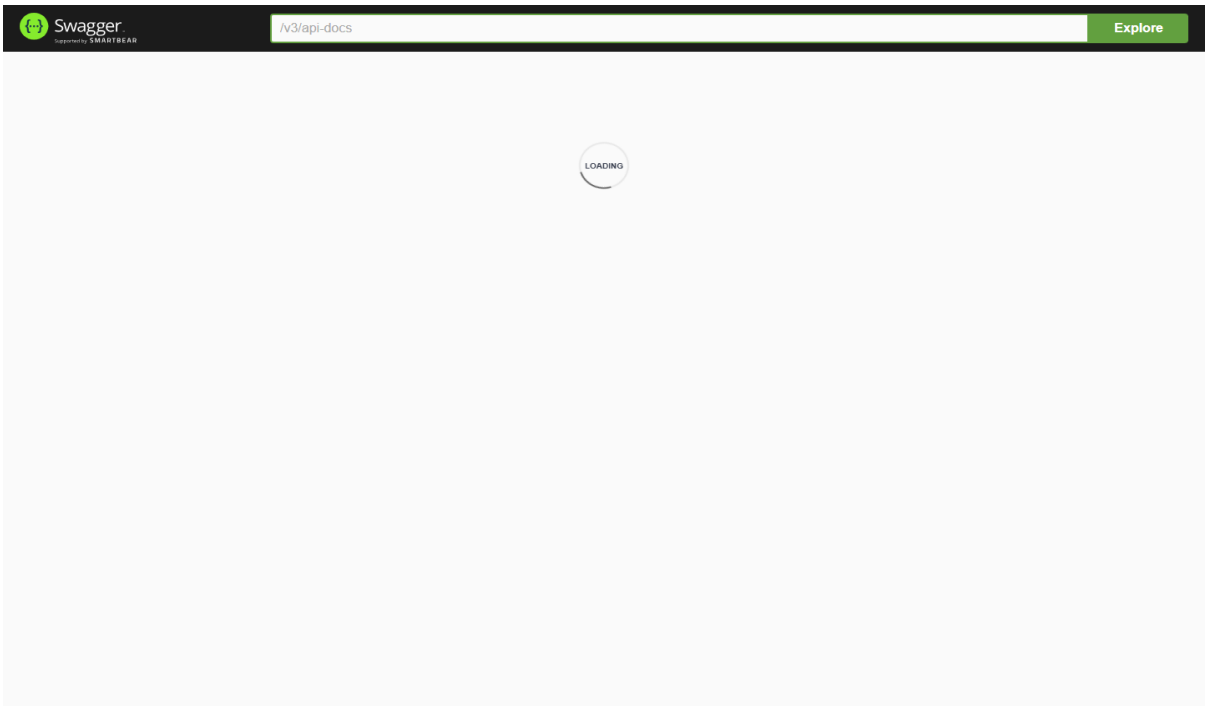


Figura 21. Backend en ejecución con Swagger UI en http://localhost:8081/swagger-ui.html

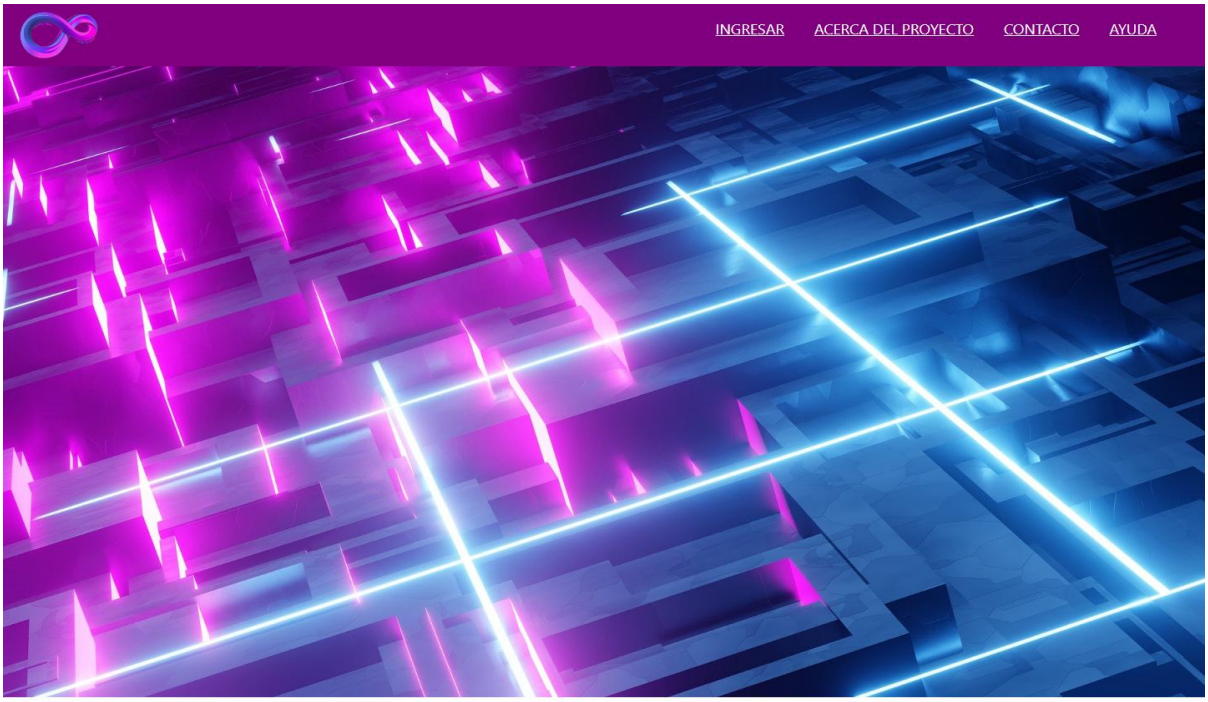


Figura 22. Frontend en ejecución en http://localhost:8080 interactuando con la API

11. Conclusión y Aprendizajes

La realización de la Práctica 3 constituyó una experiencia formativa integral en el ámbito de la Gestión de Procesos de Desarrollo de Software y la ingeniería DevOps. Se adquirieron conocimientos y destrezas prácticas determinantes para la industria actual:

1. Madurez en Pipelines CI: Se comprobó que la integración continua no es solo compilar código, sino un proceso riguroso de verificación que entrelaza pruebas unitarias, umbrales de cobertura con JaCoCo, análisis de calidad con SonarQube y escaneos de seguridad estática con Trivy. **2. Cultura de Seguridad Shift-Left:** La detección temprana y remediación de más de 140 vulnerabilidades mediante la actualización de componentes base (Tomcat, SnakeYAML, Jackson) demostró la relevancia de mantener dependencias actualizadas sin esperar a etapas tardías de producción. **3. Aislamiento e Inmutabilidad de Entornos:** El uso de Docker para orquestar contenedores de MariaDB, Jenkins y SonarQube en redes privadas garantizó entornos reproducibles, limpios y libres de contaminación de dependencias locales. **4. Rigor Metodológico en Pruebas:** Se consolidó la premisa de que la cobertura de código es una métrica cuantitativa valiosa, pero que el verdadero valor reside en la precisión de las aserciones implementadas para salvaguardar la lógica del negocio ante posibles regresiones.

En conclusión, se cumplió al 100 % con cada uno de los lineamientos de la rúbrica académica, entregando una solución robusta, moderna y completamente automatizada.